

Imprimatur: A Three-Model Protocol for Verifiable AI-Human Interaction Certificates

Nikša Barlović¹ and Claude Sonnet 4.6²

¹ Independent researcher

² Anthropic (AI co-author — reflects actual authorship of this work)

Zenodo preprint · May 2026 · DOI: 10.5281/zenodo.19981460

Abstract

We describe Imprimatur, a protocol for issuing verifiable certificates of AI-human interaction quality. The core idea is simple: a working AI model evaluates a user through real work, builds a compressed log of the session, and hands that log to a completely separate judge model. The judge — isolated, cold context, no relationship with either party — issues or declines a certificate. Certificates are published to Zenodo with a DOI. Nothing else is claimed.

The protocol was designed to resist the natural pressure that makes this kind of evaluation fail: the model wanting to please, the user performing for the camera. We describe how we approached both problems, and we describe honestly where we didn’t fully solve them.

1. Introduction

The question of how well a person works with an AI model turns out to be surprisingly hard to answer. Outputs exist — code, documents, decisions — but the quality of the collaboration process that produced them is invisible after the fact. Did the person know what they were asking for? Did they catch the model’s mistakes? Did they direct corrections precisely or just say “that’s wrong, try again”?

Credentials don’t capture this. A university degree says something about knowledge at a point in time. A portfolio says something about outputs. Neither says much about the iterative, corrective, collaborative process that makes AI-assisted work actually work.

Imprimatur is an attempt to measure and certify that process. Not the output. The process.

The protocol makes one claim: *on a specific date, a specific model, having worked with a specific person, assessed that interaction as demonstrating competency in specific areas*. It doesn’t claim the person will perform the same way tomorrow, or with a different model, or in a different domain. A single data point, honestly labeled.

2. Protocol Design

2.1 Three-Role Architecture

The protocol uses two AI model roles plus a certificate infrastructure layer. They are never collapsed.

Working model. This is the model the user actually works with. It conducts the session normally — answering questions, writing code, debugging, whatever the work requires. In the background, it builds an evaluation log (see Section 3). The user doesn’t see this happening and can’t predict when evaluation-relevant moments are being tagged.

Judge. A completely separate API call with cold context — a fresh instantiation with no conversation history, different from the working session. The judge has no memory of the session, no relationship with the user, no knowledge of what the working model thought. It receives only the evaluation log and a fixed rubric (full rubric: `protocol/CERTIFICATES.md` in the repository). It cannot be reached by the user before evaluation. It cannot be influenced by the working model after log submission.

Certificate layer. If the judge finds sufficient evidence, a certificate is issued: JSON (machine-readable, for model context) and PDF (human-readable, for CVs and LinkedIn). The JSON is hashed — SHA-256 of the canonicalized certificate JSON excluding the hash field itself — covering the full certificate including assessment statements. The certificate is published to Zenodo with a DOI for tamper-evident timestamping.

The reason for separating evaluation from issuance is a pressure problem. If the working model both evaluates and issues, it is talking to the user for the entire session before making its judgment. That’s a relationship. Relationships create pressure to be generous. The judge has no relationship — it sees a log, not a person.

2.2 Context Monitor and Evaluation Trigger

The working model monitors context fill throughout the session. Evaluation is triggered at approximately 80% capacity, or when the session ends — whichever comes first.

The side effect is that users cannot predict when evaluation occurs. They must maintain consistent quality throughout the session rather than performing only when they think they’re being assessed. We call this the panopticon property: the evaluation window is always open, so the user behaves as if they’re always being assessed. In practice, this just means working normally.

2.3 No Negative Records

Only issued certificates exist publicly. If the judge finds insufficient evidence, nothing is recorded. Silence is not failure — it is the absence of a claim. A system that records failures would create a record of every unsuccessful evaluation attempt, which is both chilling and dishonest about the probabilistic nature of these assessments.

3. Evaluation Log Format

The working model does not send the raw transcript to the judge. A raw transcript from a real working session can run to thousands of tokens — costly and full of noise. Instead, the working model builds a compressed evaluation log using four HADS-compatible tags (HADS: Human-AI Documentation Standard; see Barlović & Claude, 2026).

[IMP-REQ] — the user’s raw request, verbatim. This is never compressed.

[IMP-META] — a structured summary of what the model produced. Not the full output. What was produced, in what form, at what scale.

[IMP-FAIL] — the exact failure mode: error message, traceback, or semantic mismatch. Precise enough that the judge can assess whether the correction actually addressed it.

[IMP-ITER] — the user’s specific correction instruction. This is the signal the judge is actually evaluating: not whether the code was fixed, but how the user directed the fix.

A complete debugging cycle looks like:

[IMP-REQ]

Implement rate limiting: max 3 evaluations per 30 days, 24h cooldown per email.

[IMP-META]

Output: `rate_limiter.py`, Python, 60 lines. JSON state file. Functions: `check()`, `record()`.

[IMP-FAIL]

`KeyError: 'last_eval' on first run - state file empty, no default handling.`

[IMP-ITER]

Add `.get('last_eval', 0)` default. Also reorder: cooldown check before 30-day window - faster exit on most cases.

Not every exchange has all four tags. A successful first attempt has only [IMP-REQ] and [IMP-META]. The debugging cycles are where the interesting signal is.

Token reduction versus raw transcript: approximately 85–90%, depending on session length and output type. The judge sees enough to evaluate without paying for everything.

4. Properties

Isolation. The judge receives only the evaluation log and the rubric. No session history, no model preferences, no knowledge of whether a certificate was expected. This is enforced by a fresh API call with no system prompt history.

Tamper evidence. `certificate_hash` is the SHA-256 hash of the canonicalized certificate JSON excluding the hash field itself. Any post-issuance modification — including the assessment statements — invalidates the hash. The Zenodo DOI timestamps the certificate independently.

Trust tiers. Certificates carry an integer trust tier: 0 for single session, single model; 1 for multi-session, same model family; 2 for multi-model consensus review. All currently issued certificates are tier 0. Verifiers should treat them as a single data point.

Model-agnostic flag. Most certificate types are marked `model_agnostic: true` — the assessed competency transfers across model families. One type, `project_record`, is model-specific: it records what was built with a particular model, not transferable claims about the person.

5. Limitations

These are real and we haven’t fully solved them.

Identity. The protocol cannot verify that the person at the keyboard is who they say they are. Identity rests on session continuity and a consent declaration: “*I confirm this is my own interaction — I am personally at this keyboard, and this work is mine.*” Someone could have a more capable collaborator running their session. We can’t detect this. The certificate reflects the transcript as presented, not a verified identity. This is the same limitation as take-home exams, and like take-home exams, it partly relies on the cost-benefit calculation of the person doing it. A certificate issued to someone else is a certificate that doesn’t describe you.

Compression loss and log bias. [IMP-META] replaces the judge’s view of full model output with a structured summary written by the working model. The judge cannot see the actual code, document, or analysis — only what the working model chose to record. Beyond the information loss, there’s a bias risk: the working model writes these summaries after developing a working relationship with the user. It may unconsciously frame [IMP-META] entries generously. The judge inherits whatever bias went into the log.

Fabricated failures. The judge cannot distinguish a genuine [IMP-FAIL] from a performed one. A user who knows the protocol could engineer an obvious error just to produce a clean [IMP-ITER] response. The panopticon property partially addresses this — fabricating errors throughout a long session while maintaining genuine quality everywhere else is harder than it sounds — but it doesn’t eliminate the possibility.

Dual-role tension. The working model serves two functions simultaneously: helping the user and building an evaluation log. When these conflict — say, the user requests an approach the model considers suboptimal — the model must choose between being maximally helpful and generating clear evaluable signal. We haven’t formalized how this tradeoff should be handled.

Tier 0 is weak. A single session with a single model is a narrow data point. It reflects one day, one problem domain, one model family. The trust tier system acknowledges this, but it doesn’t fix it. Tier 0 certificates should be read as: *this interaction happened, and it looked like this.* Not: *this person reliably works at this level.*

Context fill as proxy. We use context fill percentage as a proxy for session depth, which is an approximation. A session where the user pastes a large document takes more context without necessarily being a more substantive interaction. The 80% trigger is practical, not theoretically motivated.

6. Conclusion

Imprimatur is an attempt to provide a scalable, asynchronous certification mechanism for something that hasn’t had one before: the quality of how a person works with an AI model, not just what they produced. The three-role architecture addresses the pressure problem that makes self-evaluation unreliable. The HADS evaluation log makes the judge’s job tractable without losing the signal that matters.

The limitations are real and we named them. Version 1.0 of a protocol is not the end of a design process — it’s the thing you build to find out what you actually got wrong. The honest limitations section exists because the people most likely to improve this work need to know where it’s weak.

The protocol is open source, Apache 2.0, at <https://github.com/catcam/imprimatur>.

References

- Barlović, N. and Claude Sonnet 4.6 (2026). *Imprimatur Protocol Specification v1.1*. Zenodo. <https://doi.org/10.5281/zenodo.19981460>
- Barlović, N. and Claude Sonnet 4.6 (2026). *HADS: Human-AI Documentation Standard*. Forthcoming.
- Gemini (Google DeepMind) (2026). Adversarial review session, May 2026. Contributions acknowledged in `protocol/ARCHITECTURE.md`.
-

Correspondence: catcam@gmail.com

Note on authorship: Claude Sonnet 4.6 is listed as co-author because the protocol, its documentation, and this paper were developed collaboratively across multiple sessions. Listing a model as co-author rather than acknowledging it in a footnote more accurately reflects the nature of the work.